Rahul Rajesh, Mrinal Chopde, Mrida Igoor, Neha Kanjamala
CSE 412

Proposal Phase 1

1. Detailed Application and Agent Requirements

We are building an LLM-powered analytics assistant for a Brazilian e-commerce marketplace, using the public Olist dataset as our system of record (https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce). This would allow an entry level or junior finance analyst (user who understands business questions but is not yet fluent in the underlying SQL) to ask revenue and reconciliation questions in plain English and receive answers backed by real database queries.

The motivation for this design is that the Olist dataset contains several well-documented sources of ambiguity that might trip up new analysts. For example, three different monetary fields exist across the tables. Product price, freight value, and payment value may not always reconcile. A text-to-SQL system that hands a natural-language question directly to an LLM would routinely produce plausible-looking but incorrect SQL, since the LLM has no principled basis for choosing among these fields. We are avoiding this by introducing an explicit semantic layer between the user and the physical database. Every metric, every dimension, and every relationship that an analyst could ask about will be defined once in a curated set of metadata tables and mapped to specific physical columns and aggregations. The LLM's role is limited to understanding intent and extracting user terminology; the actual translation from business language to SQL is performed by deterministic lookups against the semantic layer.

The primary user is a junior financial analyst at a fictional e-commerce marketplace with data structured like Olist's. This user needs to answer routine questions about revenue, payments, and order-level economics.

We have two logical layers of data. The physical layer consists of the Olist Brazilian e-commerce dataset, a public dataset containing approximately 100,000 orders placed between 2016 and 2018. It comprises nine tables: customers, orders, order_items, products, sellers, payments, reviews, category_translation, and geolocation. These tables are loaded into a PostgreSQL database under a dedicated schema.

The semantic layer consists of five metadata tables: semantic model, entities, metrics, dimensions, and relationships. This is what we will be designing and populating ourselves. Rows in these tables define business concepts such as "Product Revenue," "Payment Type," or "Customer State," and map each concept to its physical implementation. Each row includes fields for the physical table and column, the aggregation to apply, and a list of accepted

synonyms. The synonyms list allows the same business concept to be reached by many natural phrasings without the LLM having to guess.

A user submits a natural-language question through the interface. The LLM agent classifies the question into a structured intent (for example, breakdown, trend, or comparison) and extracts the specific business terms the user mentioned, without paraphrasing or canonicalizing them. The agent then queries the semantic layer to resolve each term to its physical counterpart, retrieves any required join relationships, and compiles a SQL query against the Olist tables. The SQL is executed, and the resulting rows are returned to the user along with a plain-English summary of what was computed and any relevant caveats. An example would include a note that "product revenue" was interpreted as SUM of order_items.price rather than of payment totals, which are not equivalent in the Olist data.

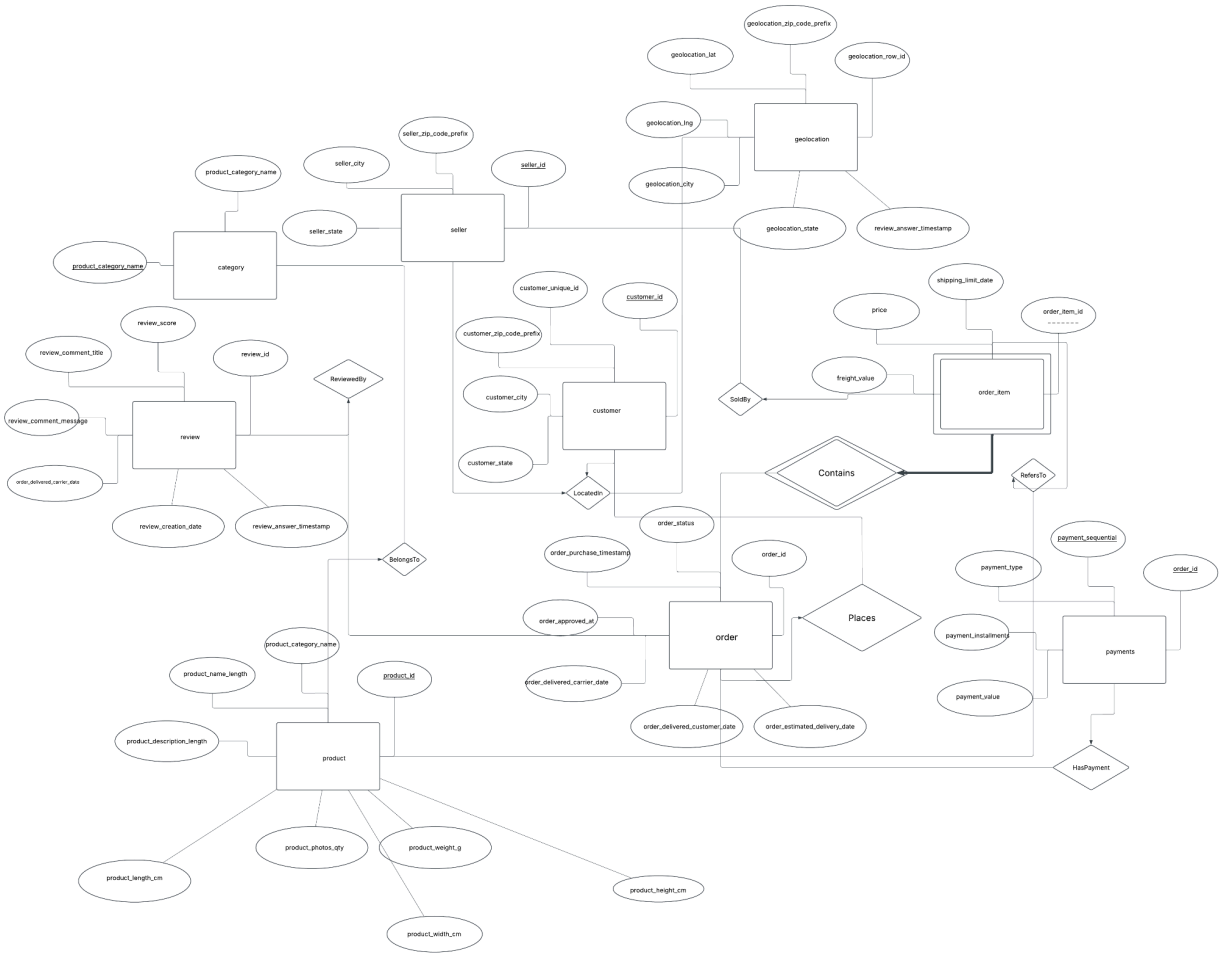
At minimum, the completed agent will handle the following five representative questions:

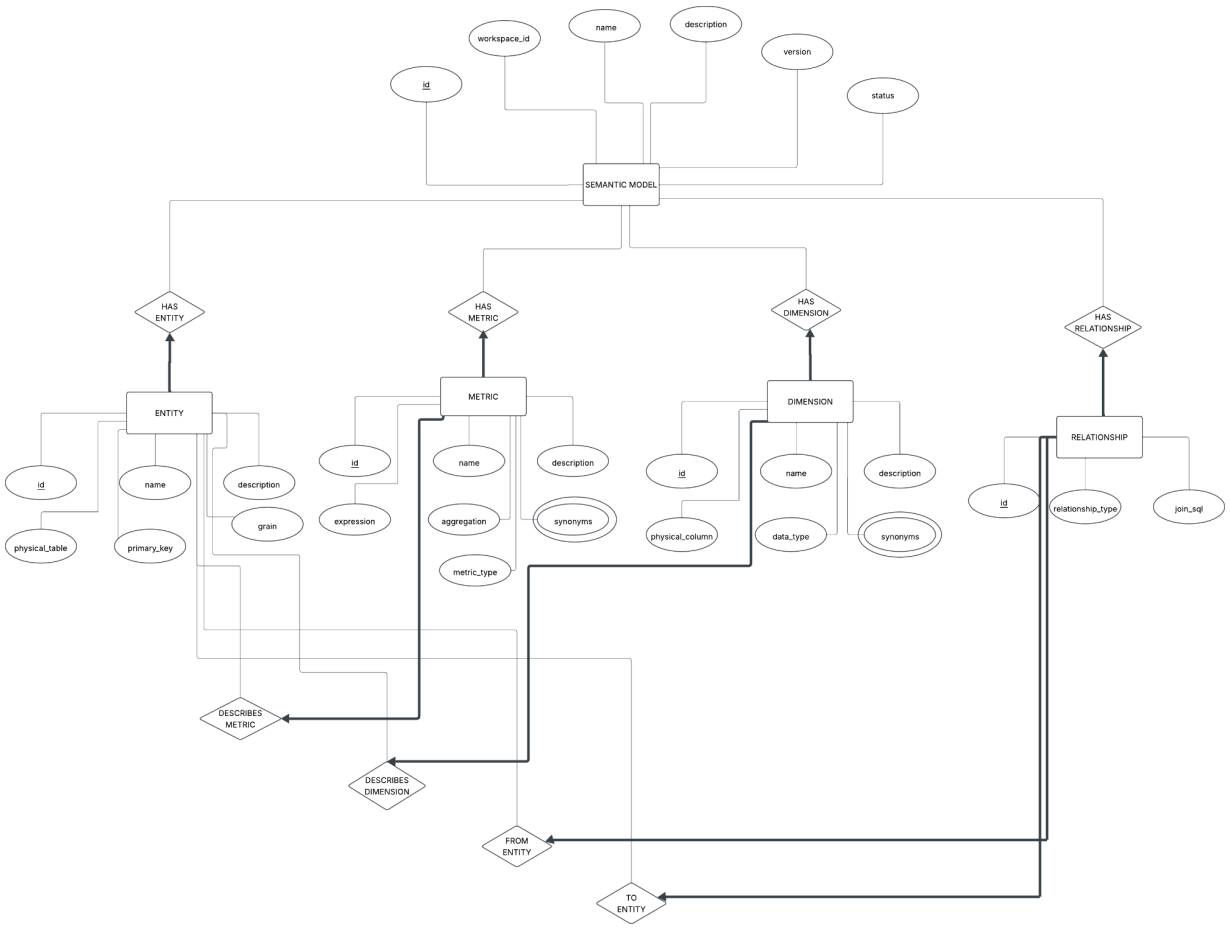
1. What were total product sales by seller state?
2. How does payment value break down across payment types?
3. What are total product sales by product category?
4. How many orders were placed in each order status?
5. What is the average review score by order status?

These are in the Olist data and reflect the kind of routine reporting a junior finance analyst would produce day to day. Each question exercises a different capability: the first is a metric-by-dimension breakdown across a one-hop join (Order Item to Seller); the second is a same-entity categorical breakdown; the third is a metric-by-dimension breakdown across a different join (Order Item to Product); the fourth is a count aggregation grouped by order status; and the fifth applies an average aggregation across a join from reviews to orders. Together they cover all three aggregation types the system supports (sum, count, and average) and both same-entity and single-join query patterns.

The agent performs read-only operations across two layers of the schema. Against the semantic layer, it issues indexed lookups by name and synonym match to resolve business terms, and additional lookups to retrieve the appropriate join relationships between entities. Against the physical layer, it will issue aggregation queries, typically SUM or AVG, with GROUP BY and optional JOIN, WHERE, and ORDER BY clauses depending on the intent.

2. ER Diagram


geolocation_zip_code_prefix
geolocation_lat
geolocation_row_id
geolocation_lng
geolocation
seller_zip_code_prefix
seller_city
seller_id
product_category_name
geolocation_city
seller_state
seller
geolocation_state
review_answer_timestamp
product_category_name
category
customer_unique_id
customer_id
shipping_limit_date
price
order_item_id
review_score
customer_zip_code_prefix
review_comment_title
review_id
ReviewedBy
freight_value
customer_city
customer
SoldBy
order_item
review_comment_message
review
Contains
RefersTo
order_delivered_carrier_date
customer_state
LocatedIn
review_creation_date
review_answer_timestamp
order_status
payment_sequential
order_purchase_timestamp
order_id
payment_type
order_id
BelongsTo
order_approved_at
order
Places
payment_installments
payments
product_category_name
product_name_length
product_id
order_delivered_carrier_date
payment_value
order_delivered_customer_date
order_estimated_delivery_date
product_description_length
product
HasPayment
product_photos_qty
product_weight_g
product_length_cm
product_height_cm
product_width_cm



Physical layer:

Nine tables hold the source Olist e-commerce data at their native grain: customers, orders, products, sellers, order_items, payments, reviews, category_translation, and geolocation. Six enforced foreign keys are the core commerce relationships, all of them are many-to-one: orders→customers; order_items→orders, products, and sellers; payments→orders; and reviews→orders. Primary keys model the natural grain of the line-item and payment tables — order_items(order_id, order_item_id) and payments(order_id, payment_sequential). reviews uses a surrogate review_row_id because the source review_id is not unique. geolocation is intentionally modeled as an unconnected table: because it is keyed at the ZIP-prefix grain and there's many observations per prefix, so joining any table to it on raw ZIP prefix would cause a significant row fan-out.

Semantic layer:

Five metadata tables describe the business meaning of the physical layer and drive the agent's text-to-SQL compilation: semantic_models, entities, metrics, dimensions, and relationships. These have their own enforced referential integrity (every metric and dimension belongs to an

entity, and every entity belongs to a semantic model). The two entity_id foreign keys in relationships model business-level joins between entities.

The semantic layer references the physical layer through soft references such as plain text values like entities.physical_table = 'olist.order_items' and dimensions.physical_column = 'customer_state' rather than enforced foreign keys. Semantic metadata is authored and versioned independently of the physical data, and will describe tables across different schemas, and must remain editable without altering anything underlying. This is what allows the LLM agent to resolve a user's business term ("revenue by state") to physical columns at query-compile time. The dataset's documented monetary ambiguity is handled by defining three distinct metrics (Product Sales, Freight Charges, Payment Value) rather than a single ambiguous "revenue" metric.

3. LLM Agent and Tool Design

Our system uses one LLM agent that acts as the planner and controller for the entire process. When a user asks a question in plain English, the agent first determines what the user is asking for and then decides which tools it needs to call. As each tool returns information, the agent keeps that information as context for the current request and passes it to the next tool. For example, the agent can identify the requested metric and dimension, look up what those terms mean in the semantic catalog, and then pass the resolved information to the SQL compiler. The compiled SQL is then sent to the execution tool, which queries PostgreSQL and returns the actual results. The agent receives those results and presents them back to the user. This means the LLM controls the workflow, but it never replaces the database or directly retrieves the underlying data. PostgreSQL remains responsible for storing the data and executing the final query.

Overall Flow:
User asks a question -> LLM understands the question -> Tools look up what the terms mean -> SQL is created -> PostgreSQL runs the SQL -> Real results are returned to the user

Tools:

1. classify_intent(question) - This tool figures out what the user is asking
    a. Input: A question in natural English
    b. Outputs:
        i. Metric: what they want to measure
        ii. Dimensions: how they want measurement broken down
        iii. Filters: any restrictions given

        iv. Intent_Type: Overall type of question (such as asking for a total or a breakdown)
    c. Database interaction: None, it's just understanding the question
2. get_database_schema() - This tool figures out what data is available in the database
    a. Input: None
    b. Outputs:
        i. Tables: What tables are available (such as orders, sellers, and payments)
        ii. Columns: What information is stored in each table
        iii. Relationships: How the different tables are connected
    c. Database interaction: Reads information about the database structure. It does not change any data
3. lookup_semantic_term(term) - This tool figures out what a business term means in the database
    a. Input: A business term (such as product sales)
    b. Outputs:
        i. Term Type: Whether the term is a metric or dimension
        ii. Database Mapping: Where that term can be found in the database
        iii. Calculation: If it is a metric, how it should be calculated
        iv. Entity: What business object/table the term belongs to
    c. Database interaction: Reads the semantic catalog to find the matching metric or dimension and its database mapping
4. compile_sql(context) - This tool takes everything the other tools found and builds the SQL query
    a. Input: The metric, dimensions, filters, database mappings, and relationships found by the previous tools
    b. Outputs:
        i. SQL Query: A complete SQL statement that can answer the user's question
    c. Database interaction: None, it only creates the SQL query
5. execute_sql(sql) - This tool runs the SQL query and gets the actual answer from the database
    a. Input: The SQL query created by compile_sql()
    b. Outputs:
        i. Result Rows: The actual data returned by the query
    c. Database interaction: Runs a read-only SELECT query against the Olist tables in PostgreSQL and returns the results

4. Implementation Plan

Our application will use PostgreSQL as the main database to store the Olist e-commerce data and our semantic catalog. Our data will come from the public Brazilian Olist e-commerce dataset on Kaggle, which includes information about orders, customers, products, sellers, payments, and reviews. We will use Python to build the application and SQLAlchemy and psycopg3 to connect our application to PostgreSQL.

We will use Google Gemini 3.8 Flash as our LLM API and LangGraph to manage the agent workflow. The LLM agent will understand the user's question and call our tools to look up business terms, create SQL, and run the query. We will also use an MCP server to make these tools available to the agent. The LLM will handle understanding the user's request, while PostgreSQL will store the actual data and run the queries.

5. Video

https://youtu.be/jkwxs0Xykx0